

HOTEL RESERVATION AND MANAGEMENT SYSTEM

1. AIM

Java Programming – Two-Page Write-Up

To design and implement a Java-based Hotel Reservation and Management System that automates guest registration, room management, room reservation, check-in/check-out, billing and payment processing. The system applies Java OOP concepts, collections, multithreading and synchronization, exception handling, AWT/Swing GUI, I/O and serialization, and JDBC with MySQL.

2. OBJECTIVES / PROCEDURE

- Identify entities such as Guest, Room, RoomType, Reservation, Service, Payment and CheckInCheckOut.
- Create classes using encapsulation, constructors, inheritance, polymorphism, method overloading, packages and interfaces.
- Store room and reservation information using ArrayList, HashMap, HashSet, generics and iterators.
- Search rooms by type, date, price and availability; support reserve, modify and cancel operations.
- Use synchronized booking logic to prevent double booking during simultaneous requests.
- Provide a Swing/AWT interface with fields, buttons, tables, menus, dialogs and event handling.
- Validate dates, room availability, cancellations and payments, and handle database errors with exceptions.
- Connect to MySQL through JDBC and perform INSERT, SELECT, UPDATE and DELETE operations.
- Calculate bills from room charges, days, services, taxes and discounts; store payment details.
- Use I/O and serialization for reservation/billing storage and backup; generate management reports.

3. ALGORITHM

1. Start the system and initialize room, guest, reservation and service collections.
2. Establish a JDBC connection with MySQL.
3. Display the GUI/menu and accept validated guest details.
4. Search for suitable rooms using type, date, price and availability.
5. When a reservation is requested, enter the booking operation inside a synchronized section.
6. Recheck availability; if available, reserve the room and create the reservation, otherwise report a conflict.
7. Process modification/cancellation and update room and reservation status.
8. During check-in/check-out, update guest and room status.
9. Calculate room charges, services, tax and discounts; validate and store payment.
10. Save information using I/O/serialization, generate reports, handle exceptions, close resources and stop.

4. PSEUDOCODE

```
BEGIN
  Initialize rooms and reservations
  Connect to MySQL using JDBC
  Read and validate guest details Display
  available rooms
  Read room number and number of days
  Find selected room

  IF room not found OR days <= 0 Display
    error
  ELSE
    SYNCHRONIZED booking
      IF selected room is unavailable Display
        "Room already reserved"
      ELSE
        Reserve room
        Create reservation
        Add reservation to collection/database
        Calculate bill = price × days + 12% tax Display
        reservation and bill
      END IF
    END SYNCHRONIZED
  END IF

  Save required data
  Generate reports
  Close database/resources
END
```

5. IMPLEMENTATION

The implementation uses three main classes—**Guest**, **Room** and **Reservation**—and a **HotelReservationSystem** driver class. Guest data is encapsulated through private fields and constructors/getters. Room stores room number, type, price and availability. The synchronized **reserve()** and **release()** methods protect room state. The reservation generates a unique reservation ID and calculates the bill using room price, number of days and 12% tax.

Core Java Implementation

```
class Guest {
    private int guestId; private
    String name, phone;

    Guest(int id, String name, String phone) {
        this.guestId = id;
        this.name = name;
        this.phone = phone;
    }
}

class Room {
    private int roomNo;
    private String type;
    private double price;
    private boolean available = true;

    public synchronized boolean reserve() { if
        (!available) return false; available =
        false;
        return true;
    }

    public synchronized void release() {
        available = true;
    }
}

static synchronized boolean makeReservation( Guest
    guest, Room room, int days) {
    if (!room.isAvailable()) return false;

    if (room.reserve()) {
        Reservation r = new Reservation(guest, room, days);
        reservations.add(r);
        r.display();
        return true;
    }
    return false;
}
```

6. BILLING IMPLEMENTATION

```
public double calculateBill() {
    double roomCharge = room.getPrice() * days;
    double tax = roomCharge * 0.12;
    return roomCharge + tax;
}
```

For the sample test, a Double room costs Rs.2500 per day and is booked for 3 days. Room charge = Rs.7500; 12% tax = Rs.900; total bill = **Rs.8400.00**.

7. SAMPLE INPUT

Guest ID: 101 Guest Name: Pranav Phone: 9876543210 Room
Number: 102 Number of Days: 3

8. EXPECTED OUTPUT

Reservation ID: 1001
Guest: Pranav
Room No: 102
Room Type: Double
Days: 3
Bill: Rs.8400.00
Reservation successful.

9. RESULT

The Java program successfully demonstrates guest registration, room availability checking, reservation creation, synchronized booking logic and billing with tax. The complete assignment specification can be extended with Swing/AWT, MySQL/JDBC CRUD operations, I/O and serialization, concurrent booking simulation, exception handling and report generation.

10. CONCLUSION

Thus, a structured Hotel Reservation and Management System is designed using object-oriented programming, collections, synchronization and billing logic. The implementation provides a foundation that can be extended with GUI, database operations, file handling, validation, reporting and recovery features.

HOTEL RESERVATION AND MANAGEMENT SYSTEM

PSEUDOCODE

PSEUDOCODE

```
BEGIN Hotel Reservation System
Initialize rooms and reservations
Create rooms:

101 → Single → Rs.1500
102 → Double → Rs.2500
103 → Deluxe → Rs.4000
Read Guest ID, Name and Phone
Create Guest object
Display available rooms
Read Room Number and Days
Find selected room
IF room not found
Display "Room not found"
ELSE IF days <= 0
Display "Invalid number of days"
ELSE
SYNCHRONIZED booking
IF room unavailable
Display "Room is already reserved"
ELSE
Reserve room
Create Reservation
Generate Reservation ID
Add reservation to collection

roomCharge ← price × days

tax ← roomCharge × 0.12

total ← roomCharge + tax
Display reservation and bill
Display "Reservation successful"
END IF
END SYNCHRONIZED
END IF
Save required information
Generate reports when required
Close resources
END
```

BOOKING & BILLING PSEUDOCODE

```
FUNCTION makeReservation(guest, room, days)
SYNCHRONIZED
IF room is unavailable
RETURN false
END IF
IF room.reserve() succeeds
r ← new Reservation(guest, room, days)
Add r to reservations
Display r
RETURN true
END IF
RETURN false
END SYNCHRONIZED
END FUNCTION
FUNCTION calculateBill()

roomCharge ← room.price × days

tax ← roomCharge × 0.12
RETURN roomCharge + tax
END FUNCTION
```

Page 2 of 2

HOTEL RESERVATION AND MANAGEMENT SYSTEM

JAVA IMPLEMENTATION – TWO COLUMNS

Guest & Room Classes

```
import java.sql.*;
import java.util.*;
class Guest {
private int guestId;
private String name, phone;
Guest(int id, String name, String phone) {
guestId=id; this.name=name; this.phone=phone;
}
public int getGuestId(){return guestId;}
public String getName(){return name;}
public String getPhone(){return phone;}
}
class Room {
private int roomNo;
private String type;
private double price;
private boolean available;
Room(int no,String type,double price){
roomNo=no; this.type=type;
this.price=price; available=true;
}
public int getRoomNo(){return roomNo;}
public String getType(){return type;}
public double getPrice(){return price;}
public boolean isAvailable(){return available;}
public synchronized boolean reserve(){
if(!available)return false;
available=false; return true;
}
public synchronized void release(){available=true;}
}
```

Reservation Class

```
class Reservation {
private static int nextId=1001;
private int reservationId;
private Guest guest;
private Room room;
private int days;
Reservation(Guest guest,Room room,int days){
reservationId=nextId++;
this.guest=guest; this.room=room;
this.days=days;
}
public double calculateBill(){
double roomCharge=room.getPrice()*days;
double tax=roomCharge*0.12;
return roomCharge+tax;
}
public void display(){
System.out.println("Reservation ID : "+reservationId);
System.out.println("Guest : "+guest.getName());
System.out.println("Room No : "+room.getRoomNo());
}
```

```

System.out.println("Room Type : "+room.getType());
System.out.println("Days : "+days);
System.out.printf("Bill : Rs. %.2f\n",calculateBill());
}
}

```

Hotel Reservation System

```

public class HotelReservationSystem {
    static ArrayList<Room> rooms=new ArrayList<>();
    static ArrayList<Reservation> reservations=new ArrayList<>();
    static synchronized boolean makeReservation(
        Guest guest,Room room,int days){
        if(!room.isAvailable()) return false;
        if(room.reserve()){
            Reservation r=new Reservation(guest,room,days);
            reservations.add(r);
            r.display();
            return true;
        }
        return false;
    }

    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        rooms.add(new Room(101,"Single",1500));
        rooms.add(new Room(102,"Double",2500));
        rooms.add(new Room(103,"Deluxe",4000));
        System.out.println("=== HOTEL RESERVATION SYSTEM ===");
        System.out.print("Enter Guest ID: ");
        int id=sc.nextInt(); sc.nextLine();
        System.out.print("Enter Guest Name: ");
        String name=sc.nextLine();
        System.out.print("Enter Phone: ");
        String phone=sc.nextLine();
        Guest guest=new Guest(id,name,phone);
        System.out.println("Available Rooms:");
        for(Room r:rooms)
            if(r.isAvailable())
                System.out.println(r.getRoomNo()+" - "+
                    r.getType()+" - Rs."+r.getPrice());
        System.out.print("Enter Room Number: ");
        int roomNo=sc.nextInt();
        System.out.print("Enter Number of Days: ");
        int days=sc.nextInt();
        Room selected=null;
        for(Room r:rooms)
            if(r.getRoomNo()==roomNo){
                selected=r; break;
            }
        if(selected==null)
            System.out.println("Room not found.");
        else if(days<=0)
            System.out.println("Invalid number of days.");
        else if(makeReservation(guest,selected,days))
            System.out.println("Reservation successful.");
        else
            System.out.println("Room is already reserved.");
        sc.close();
    }
}

```

✓ No Limits ✓ Accurate Reports



1. AIM
HOTEL RESERVATION AND
MANAGEMENT SYSTEM
Java Programming – Two-Page
Write-Up

Page 1

To design and implement a Java-
based Hotel Reservation and
Management System that
automates guest registration, room
management, room reservation,
check-in/check-out, billing and
payment processing. The system
applies Java OOP concepts,

5376 / 10,000 chars - 756 Words

Tx



Check Plagiarism via URL

Exclude URLs



Check Plagiarism →



0%

Plagiarized

Exact Match 0%

Partial Match 0%

100%

Unique
Original Content

Plagiarism Checker

Check Grammar